

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Apple M4 — Monte Carlo: 5.27× (10 threads); Heat float64: 2.53× (4 threads); Heat float32: 3.84× (4 threads).

Student Name: Symbat Aldanova

Student ID: 230103170

Hardware / CPU: Apple M4

Date: 2 October 2026

Section: [01N] [03 P]

Threading backend: OpenMP (omp)

Measurement method: Median of three runs, excluding JIT compilation.

Benchmark	Threads	Array size / samples	Time (s)	Speedup	Efficiency
Monte Carlo	1	120,000,000	0.6303	1.00×	100.0%
Monte Carlo	2	120,000,000	0.3218	1.96×	97.9%
Monte Carlo	4	120,000,000	0.1635	3.85×	96.3%
Monte Carlo	8	120,000,000	0.1290	4.88×	61.1%
Monte Carlo	10 (maximum)	120,000,000	0.1196	5.27×	52.7%
Mandelbrot — rows	10	2500 × 2500	3.1797	N/A	N/A
Mandelbrot — columns	10	2500 × 2500	3.1861	N/A	N/A
Heat — float64	10	1500 × 1500 × 300	0.1346	N/A	N/A
Heat — float32	10	1500 × 1500 × 300	0.0582	N/A	N/A

Challenge 1

A. Baseline and maximum-thread runtime

The one-thread baseline was **0.6303 seconds**. At the maximum of 10 threads, runtime was **0.1196 seconds**, giving a speedup of **5.27×**.

B. Why is the warm-up required?

Numba compiles the function during its first call. The warm-up completes compilation before timing starts. Otherwise, the measured time would include compilation and would not represent execution time alone.

C. Did efficiency remain at 100%?

No. Efficiency decreased from **97.9% at 2 threads** to **52.7% at 10 threads**. Possible reasons include thread management, synchronization, shared hardware resources, and differences in core performance. These measurements show reduced scaling, but they do not identify the exact contribution of each factor.

D. Why is the counter safe?

Numba recognizes `inside_circle += 1` as a sum reduction. Each thread accumulates a private count, and the counts are combined after the parallel loop. This is equivalent to the OpenMP construct `reduction(+:inside_circle)`.

Challenge 2

A. Which decomposition was faster?

Row decomposition took **3.1797 seconds**, while column decomposition took **3.1861 seconds**. Rows were approximately **0.2% faster**. The difference is too small to establish a reliable performance advantage from these measurements alone.

B. How does memory layout affect cache usage?

NumPy stores these arrays in C-contiguous, row-major order. Consecutive columns within one row are adjacent in memory. The row version therefore writes consecutive elements and uses cache lines efficiently.

The column version still writes `img[r, c]`, but changes `r` in the inner loop. Consecutive writes are separated by a full row, which generally gives poorer memory locality.

C. Why does load imbalance occur?

Rows near the outer regions often require few iterations, while central rows contain more points that reach the iteration limit. A thread assigned outer rows may finish early and wait while another thread continues processing central rows.

Dynamic scheduling gives threads smaller chunks of work. When a thread finishes a chunk, it receives another one, reducing idle time. The supplied benchmark compares axes; it does not explicitly enable or measure dynamic scheduling.

D. Visual artifact

Attach `mandelbrot_output.png` from the `openmp_results` folder.

Challenge 3

A. Runtime and throughput

At 10 threads:

Precision	Runtime	Throughput
float64	0.1346 s	5,01 4.28 Meg acell s/s
float32	0.0582 s	11,6 00.5 5 Meg acell s/s

Throughput follows the task sheet's formula, which counts the full grid.

B. Effect of float32

At 10 threads, the runtime improvement factor was:

$$\frac{T_{float64}}{T_{float32}} = \frac{0.1346155}{0.0581869} \approx 2.31$$

The float32 version was **2.31× faster**, reducing runtime by approximately **56.8%**. Smaller arrays reduce data traffic and improve cache usage. Changes in arithmetic and vectorization may also contribute.

C. Memory roofline explanation

Stencil computation performs relatively few arithmetic operations for the data it accesses. Once memory bandwidth is saturated, additional cores cannot obtain data proportionally faster. Synchronization and thread scheduling can also increase overhead.

In my results, both precisions performed best at **4 threads**. Increasing the thread count to 8 or 10 made execution slower. This is consistent with shared-resource limits and parallel overhead, although these timings alone do not prove that RAM bandwidth was saturated.

Best Heat results

Precision	Best thread count	Runtime	Speedup versus 1 thread
float64	4	0.0789 s	2.53×
float32	4	0.0484 s	3.84×